Gatekeeper: A Rate-Limiting API Gateway in Go

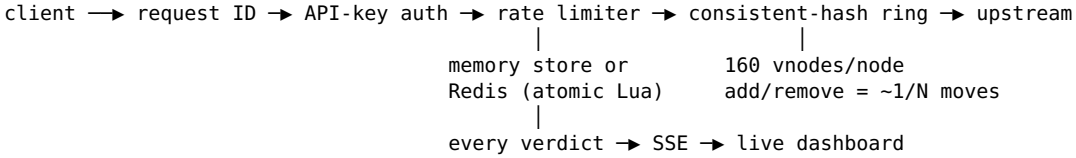
Mahesh Thakur, IIT Patna (Mathematics and Computing)

October 2026

Overview

Gatekeeper is a working rate-limiting API gateway: the component that sits in front of a service and decides, thousands of times per second, which requests pass and which are throttled. It implements the two standard limiting algorithms (token bucket, sliding window) both in memory and as atomic Redis Lua scripts, routes across upstreams with consistent hashing, authenticates clients with API keys, and shows every verdict live on a dashboard.

Architecture



The two algorithms, and why both

Token bucket refills at a steady rate up to a burst cap. It tolerates legitimate bursts while enforcing a sustained rate. **Sliding window** (weighted previous-window estimate) fixes the fixed-window edge stampede: at the window boundary at most a couple of requests squeeze through, never a full second burst. Both are written as pure state transitions and tested against a virtual clock; the Redis versions execute the identical math in Lua so replicas share one limiter state atomically. Parity between the two stores is tested against miniredis with a fast-forwarded clock.

Consistent hashing

The ring (Karger et al., 1997; 160 virtual nodes per upstream, md5 points) keeps key-to-upstream mapping stable: tested over 30,000 keys, three nodes hold ~33% each, and adding a fifth node moves ~20% of keys and never more than 35% - the property that makes horizontal scaling cheap.

Measured performance

From `cmd/bench` (raw record in `docs/benchmarks.json`): **0.47 µs per limiter decision** (~2.1M decisions/sec), and **4,391 req/s through the full proxy path** (auth + limit + route + proxy) with **p99 latency 0.84 ms**, measured over 2,000 sequential keep-alive requests. The single-connection method is deliberate and documented: the measurement sandbox throttles concurrent loopback connections.

Failure behavior, tested

Throttled clients receive HTTP 429 with `Retry-After` and `X-RateLimit-*` headers, and rejected requests never reach the upstream (asserted). Clients are isolated: one client exhausting its budget never affects another. A slow dashboard subscriber drops events rather than blocking the gateway, proven under the race detector. If Redis is unavailable the gateway fails open and logs - the standard gateway choice, documented and configurable.

Testing

`go test -race ./...`: algorithm correctness on a virtual clock, Redis/memory parity, ring distribution

and minimal remapping, auth, headers, per-key isolation, stats, and backpressure safety. CI runs the same on every push.